

Software Architecture Design Report
Smart AI-Based Urban Street Lighting Management
System

CO2 ASSESSMENT TOOL 2
ARCHITECTURE DESIGN ASSIGNMENT

NAME : KEERTHANA S
REG NO: 192471047
COURSE NAME : SOFTWARE ENGINEERING
COURSE CODE : CSA1017

1. System Requirement Analysis

1.1 System Objectives

The proposed system replaces fixed-schedule municipal street lighting with an adaptive, AI- and IoT-driven platform. Its objectives are to continuously monitor streetlight status, automatically detect faults, optimize brightness using AI based on real-world conditions, reduce electricity consumption, predict maintenance needs, generate energy usage reports, improve public safety, and support broader smart-city initiatives.

1.2 Functional Requirements Influencing Architecture

- Continuous, near-real-time ingestion of status/telemetry data from thousands of distributed IoT streetlight sensors.
- Automated fault detection and alert notification to maintenance staff with minimal latency.
- AI-driven brightness optimization that consumes multiple live data feeds (traffic, weather, pedestrian movement).
- Predictive maintenance analytics over historical sensor and fault-history data.
- Centralized dashboard aggregating city-wide status for administrators, plus role-differentiated mobile and public-facing interfaces.
- Periodic report generation and export, and integration with external Smart City / GIS / ERP systems.

1.3 Non-Functional Requirements Influencing Architecture

Quality Attribute	Architectural Implication
Scalability	Must independently scale ingestion of tens of thousands of IoT devices and city-wide expansion without redesign — favors loosely coupled, horizontally scalable services.
Performance	Sensor data must be processed and reflected on the dashboard within seconds — favors asynchronous, event-driven data flow over synchronous chains.
Reliability	Faults must be detected and alerted even under partial network/service failure — favors decoupled services and durable message queues.

Quality Attribute	Architectural Implication
Availability	Monitoring and alerting must run 24/7 — favors redundant, independently deployable services rather than a single monolithic process.
Security	Role-based access, encrypted telemetry, and secure device authentication are required — favors a centralized API Gateway enforcing auth/policy.
Maintainability	New AI models, sensor types, or reporting features must be added with minimal disruption — favors modular services with clear boundaries.

Together, these requirements point to a system that is distributed by nature (edge devices generating continuous streams), bursty and asynchronous in data flow, and composed of functionally independent capabilities (monitoring, fault detection, AI optimization, prediction, reporting, notification) that evolve at different rates.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

A Hybrid Architecture combining **Microservices Architecture** with an **Event-Driven Architecture** for backend/cloud processing, an **Edge Computing layer** for IoT data pre-processing, and a centralized **API Gateway** fronting client access, is selected as the most suitable architecture for this system.

2.2 Justification

- **Microservices:** Each capability — monitoring, fault detection, AI optimization, predictive maintenance, reporting, notification, authentication, and work-order management — has distinct scaling needs, release cadence, and technology requirements (e.g., the AI service benefits from GPU-backed infrastructure while the notification service does not). Microservices let each be built, deployed, and scaled independently.
- **Event-Driven communication:** Streetlight telemetry arrives continuously and asynchronously from thousands of devices. An event bus / message broker decouples data producers (sensors) from consumers (services), allowing the system to absorb bursty IoT traffic without blocking, and enabling multiple services (fault detection, AI optimization, logging) to independently react to the same telemetry stream.

- Edge Computing: Performing local aggregation and protocol translation at edge gateways reduces bandwidth usage, filters noise before it reaches the cloud, and allows basic fault flags to be raised even during temporary cloud connectivity loss.
- API Gateway: Centralizing authentication, rate limiting, and routing for the three client types (admin dashboard, mobile app, public portal) simplifies security enforcement and shields internal service boundaries from external clients.
- Alternative styles considered: a monolithic layered architecture was rejected because it would not allow independent scaling of the AI workload versus lightweight services, and would create a single point of failure for a system that must run 24/7 across an entire city. A pure request/response client-server model was rejected because it cannot efficiently handle high-frequency, asynchronous IoT telemetry at scale.

3. Software Architecture Diagram

The diagram below illustrates the complete system structure: client applications, the API Gateway, the microservices application layer, the event bus, the AI/analytics engine, the edge/IoT layer, the data layer, and external system integrations.

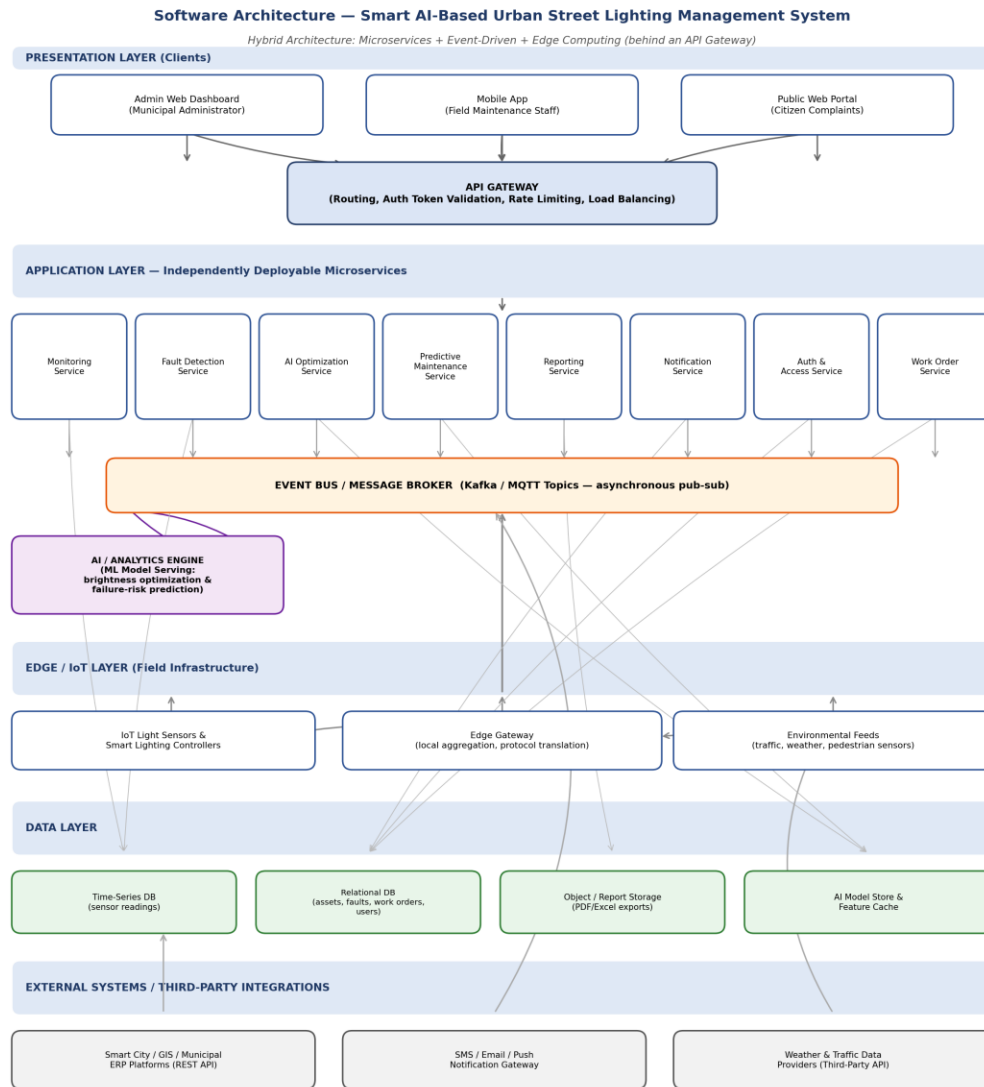


Figure 1: Proposed Hybrid Architecture (Microservices + Event-Driven + Edge Computing)

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility
Admin Web Dashboard / Mobile App / Public Portal	Client-facing applications for administrators, field staff, and citizens respectively; consume APIs exposed through the gateway.

Component	Responsibility
API Gateway	Single entry point for all client requests; performs authentication token validation, request routing, rate limiting, and load balancing across microservices.
Monitoring Service	Maintains the real-time status of every streetlight and serves current-state queries to the dashboard.
Fault Detection Service	Analyzes incoming telemetry events for anomalies and raises fault records.
AI Optimization Service	Requests brightness recommendations from the AI/Analytics Engine and issues control commands to lighting controllers via the edge gateway.
Predictive Maintenance Service	Consumes historical data to schedule preventive maintenance recommendations.
Reporting Service	Aggregates data for energy usage and cost-saving reports, exportable in PDF/Excel.
Notification Service	Delivers fault alerts and work-order updates via SMS/email/push to maintenance staff.
Auth & Access Service	Manages user identity, roles, and permissions across all client types.
Work Order Service	Tracks maintenance task lifecycle from creation to closure.
Event Bus / Message Broker	Decouples producers and consumers of telemetry and domain events (fault-detected, brightness-changed, maintenance-due) using asynchronous publish/subscribe topics.
AI / Analytics Engine	Hosts and serves the ML models used for brightness optimization and failure-risk prediction, consuming events from the bus and returning predictions.
Edge Gateway	Aggregates and pre-processes data from nearby IoT sensors, translates device protocols, and relays commands back to controllers.
Data Layer (Time-Series DB, Relational DB, Object Storage, Model Store)	Persists sensor readings, asset/fault/user records, generated reports, and trained AI models/feature data respectively.

Component	Responsibility
External Systems	Third-party Smart City/GIS/ERP platforms, SMS/email gateways, and weather/traffic data providers integrated via REST APIs.

4.2 Communication Mechanisms

- Clients communicate with backend services synchronously over HTTPS/REST through the API Gateway.
- IoT sensors and edge gateways communicate using lightweight IoT protocols (MQTT/CoAP) suited to constrained devices.
- Microservices communicate with each other asynchronously by publishing and subscribing to events on the message broker, avoiding tight coupling and cascading failures.
- The AI/Analytics Engine is invoked both via direct service calls (for on-demand predictions) and via event subscription (for batch/streaming analysis).
- Each microservice owns and accesses its relevant data store directly, following a database-per-service pattern where practical, while shared reference data (e.g., asset registry) resides in the relational database.

4.3 Overall Workflow

- 1. IoT sensors stream light intensity, power draw, and status to the edge gateway, which aggregates and forwards the data to the cloud.
- 2. Telemetry is published to the event bus, where the Monitoring, Fault Detection, and AI Optimization services independently consume it.
- 3. The Fault Detection service raises a fault event if anomalies are found; this triggers the Notification service to alert field staff and the Work Order service to create a task.
- 4. The AI Optimization service, together with the AI/Analytics Engine, computes optimal brightness using live traffic/weather/pedestrian feeds and issues a command back through the edge gateway to the lighting controller.
- 5. The Predictive Maintenance service periodically analyzes historical data to flag components at risk, feeding the Work Order service.

- 6. Administrators view aggregated results on the dashboard (via the Monitoring and Reporting services); citizens view public status and submit complaints via the portal, which also raises events for staff follow-up.

5. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Each microservice scales independently (e.g., the Monitoring and AI Optimization services can be scaled out separately from the low-traffic Reporting service). The event bus buffers load spikes from bursty IoT traffic, and edge aggregation reduces the volume reaching the cloud, allowing the system to grow from a pilot zone to city-wide coverage without redesign.
Security	The API Gateway centralizes authentication and authorization, so no internal service is directly exposed to clients. Role-based access control is enforced by the Auth & Access Service, and telemetry is encrypted in transit between edge devices and the cloud (TLS/DTLS) and at rest in the data layer.
Performance	Asynchronous event-driven communication avoids blocking calls between services, so a slow consumer (e.g., Reporting) cannot delay time-critical paths (e.g., Fault Detection). Edge pre-processing reduces latency and bandwidth for time-sensitive brightness adjustments.
Reliability	Decoupling via the message broker means a temporary outage in one service (e.g., Notification) does not block others from processing events; the broker retains and redelivers messages once the consumer recovers. Edge gateways can apply basic local rules if cloud connectivity is briefly lost.
Maintainability	Clear service boundaries mean new capabilities (e.g., a new AI model version or a new sensor type) can be developed, tested, and deployed independently without touching unrelated services, reducing regression risk and easing long-term evolution.
Availability	Independently deployable, redundant service instances behind the gateway avoid a single point of failure. Critical services (Monitoring, Fault Detection, Notification) can be replicated across availability zones to sustain the 24/7 uptime the system requires.

6. Justification of Architectural Decisions and Trade-offs

6.1 Rationale

The combination of microservices, event-driven communication, and edge computing was chosen because the problem domain is inherently distributed (thousands of geographically spread sensors), asynchronous (continuous telemetry rather than discrete user-triggered requests), and functionally heterogeneous (AI workloads, simple CRUD-style services, and real-time alerting coexist). A single architectural style alone would not comfortably satisfy all these characteristics — a purely layered monolith would hinder independent scaling and resilience, while a purely event-driven design without a gateway would complicate client-facing security and routing.

6.2 Trade-offs Considered

- Complexity vs. flexibility: Microservices introduce operational complexity (service discovery, distributed monitoring, network latency between services) compared to a monolith, but this is accepted in exchange for independent scalability and fault isolation, which are essential for a 24/7 city-wide safety system.
- Eventual consistency vs. simplicity: Event-driven communication means some services (e.g., the dashboard's fault count) may be briefly out of sync with the true system state. This is an acceptable trade-off given the near-real-time (not hard real-time) nature of the requirements, and is preferable to the tight coupling of synchronous chains.
- Edge processing cost vs. cloud bandwidth: Adding intelligence at edge gateways increases field-hardware cost and deployment effort, but significantly reduces cloud bandwidth and improves resilience to intermittent connectivity — a worthwhile trade-off for citywide IoT deployments.

6.3 Support for Future Enhancements and Long-Term Maintainability

- New services (e.g., smart parking, adaptive traffic signals) can be added as additional microservices that subscribe to the same event bus, supporting broader smart-city integration without modifying existing services.
- The AI/Analytics Engine can be upgraded or replaced with newer models independently, since it is decoupled from other services via well-defined APIs and events.

- The API Gateway provides a stable, versioned contract for client applications even as backend services evolve internally.
- Because each service owns its data, teams can migrate individual services to new technologies or scale them on different infrastructure (e.g., GPU nodes for AI) without impacting the rest of the system, supporting sustainable long-term maintenance.